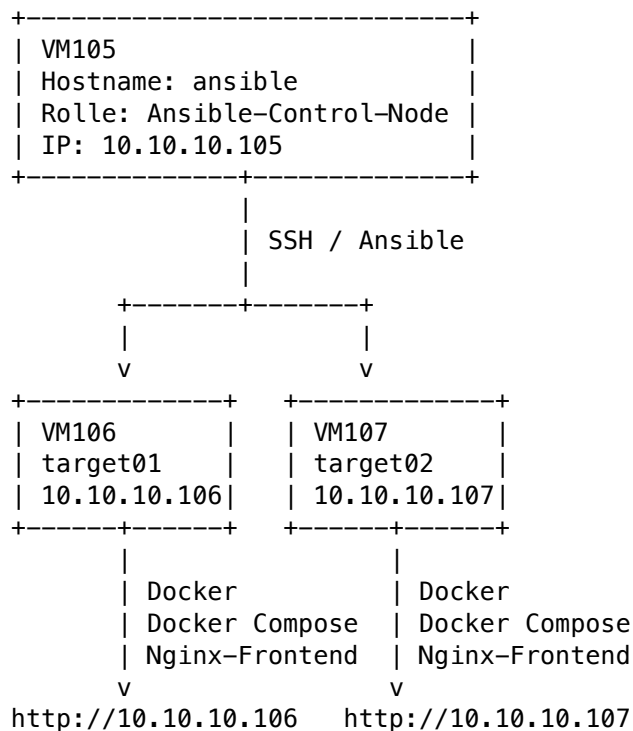


Projektstruktur

Infrastruktur

VM	Hostname	Rolle	IP-Adresse
VM105	ansible	Ansible-Control-Node	10.10.10.105
VM106	target01	Docker-Webserver 1	10.10.10.106
VM107	target02	Docker-Webserver 2	10.10.10.107

Architektur



```
ansible-project/
├── ansible.cfg
├── requirements.yml
├── inventory/
│   └── hosts.yml
```

```

├── group_vars/
│   └── webservers.yml
├── playbooks/
│   └── site.yml
├── roles/
│   ├── common/
│   │   ├── tasks/
│   │   │   └── main.yml
│   │   └── handlers/
│   │       └── main.yml
│   ├── docker/
│   │   └── tasks/
│   │       └── main.yml
│   └── webapp/
│       ├── tasks/
│       │   └── main.yml
│       ├── templates/
│       │   ├── docker-compose.yml.j2
│       │   ├── nginx.conf.j2
│       │   └── index.html.j2
│       └── handlers/
│           └── main.yml

```

Beschreibung der wichtigsten Dateien

Datei / Ordner	Beschreibung
<code>ansible.cfg</code>	Enthält die Grundeinstellungen für Ansible, zum Beispiel den Pfad zum Inventory.
<code>requirements.yml</code>	Externe Ansible-Collections (hier <code>community.docker</code>) für den Control-Node.
<code>inventory/hosts.yml</code>	Enthält die Zielsysteme <code>target01</code> und <code>target02</code> .
<code>inventory/group_vars/webservers.yml</code>	Enthält Variablen, die für beide Webserver gelten.
<code>playbooks/site.yml</code>	Haupt-Playbook, das alle Rollen ausführt.
<code>roles/common/</code>	Allgemeine Vorbereitung der Zielsysteme, zum Beispiel Paketupdates.
<code>roles/docker/</code>	Installiert Docker und Docker Compose auf den Targets.
<code>roles/webapp/</code>	Erstellt und startet die Webanwendung mit Nginx im Docker-Container.

Ziel des Deployments

Nach dem Ausführen des Ansible-Playbooks sollen beide Zielsysteme automatisch eingerichtet sein.

Auf beiden Targets wird folgendes umgesetzt:

- Docker wird installiert.
- Docker Compose wird eingerichtet.
- Eine Nginx-Webanwendung wird als Container gestartet.
- Eine einfache HTTP-Seite wird bereitgestellt.
- Die Website ist im internen Netzwerk erreichbar.

Voraussetzungen

Control-Node (ansible, VM105):

- Ansible ist installiert.
- Die benötigte Collection ist installiert:
`ansible-galaxy collection install -r requirements.yml`
- Der SSH-Key ist einmalig pro Target verteilt:
`ssh-copy-id ansible@10.10.10.106`

Targets (target01, target02):

- Betriebssystem: Debian/Ubuntu (apt).
- Dedizierter Login-User (Standard: ansible) mit sudo-Rechten (idealerweise NOPASSWD).
- Python 3 ist vorhanden (für die Ansible-Module).

Login-User ist zentral in `inventory/group_vars/webserver.yml` (`ansible_user`) einstellbar. Ohne NOPASSWD-sudo das Playbook mit `--ask-become-pass` starten.

Bereitstellung & Test-Workflow

Getestet wird **zuerst nur target01**. Erst wenn dort alles läuft, wird target02 identisch ausgerollt — Rollen und Variablen gelten für beide Hosts gleichermaßen.

1. Erreichbarkeit prüfen (nur target01)

```
ansible webserver -m ping --limit target01
```

2. Syntax-Check des Playbooks

```
ansible-playbook playbooks/site.yml --syntax-check
```

3. Deployment auf target01

```
ansible-playbook playbooks/site.yml --limit target01
```

4. Ergebnis prüfen

```
curl http://10.10.10.106
```

5. Im Browser Webseite öffnen von eigenem Rechner

```
ssh -L 8106:10.10.10.106:80 -L 8107:10.10.10.107:80 ss@100.66.18.54
```

6. Nach erfolgreichem Test: target02 ausrollen

```
ansible-playbook playbooks/site.yml --limit target02
```

(oder ganz ohne --limit für beide Hosts gleichzeitig)

Hinweis: Ein `--check`-Trockenlauf kann bei diesem Playbook fehlschlagen, weil spätere Schritte (Docker-Repo einbinden, Container starten) auf zuvor installierten Paketen aufbauen, die im Check-Modus noch nicht real vorhanden sind. Für den ersten echten Test ist `--limit target01` der richtige Weg.

Aufruf der Webanwendung

Nach erfolgreichem Deployment können die Webseiten über folgende Adressen getestet werden:

`http://10.10.10.106`

`http://10.10.10.107`

Alternativ kann der Test von der Ansible-Maschine aus mit `curl` durchgeführt werden:

```
curl http://10.10.10.106  
curl http://10.10.10.107
```

Zielbild

Das Ziel ist, dass die VM105 als zentrale Verwaltungsmaschine dient. Von dort aus werden die beiden Zielsysteme VM106 und VM107 automatisiert konfiguriert. Dadurch wird gezeigt, wie mehrere Linux-Server mit Ansible verwaltet und Docker-Anwendungen automatisiert bereitgestellt werden können.